

Q23: Discuss the reference model of full virtualization.

ANS:-

Virtualization is a technique that allows multiple operating systems and applications to run on the same physical hardware at the same time. Among the different types of virtualization, **full virtualization** is one of the most widely used models. The **reference model of full virtualization** explains how the system is organized into layers and how these layers interact to provide the illusion that each virtual machine (VM) has access to its own dedicated hardware. The important feature of this model is that the guest operating system does not need to be modified; it runs exactly as it would on a real physical computer.

Example: You can run **Windows OS inside a virtual machine** while your real computer is running **Linux OS**.

This helps us use one physical machine for **multiple purposes at the same time**, without disturbing each other.

Example: Imagine one big hotel (a physical server). Virtualization divides it into many rooms (virtual machines). Each guest (user) feels like they have their own private space, even though they're all inside the same hotel.

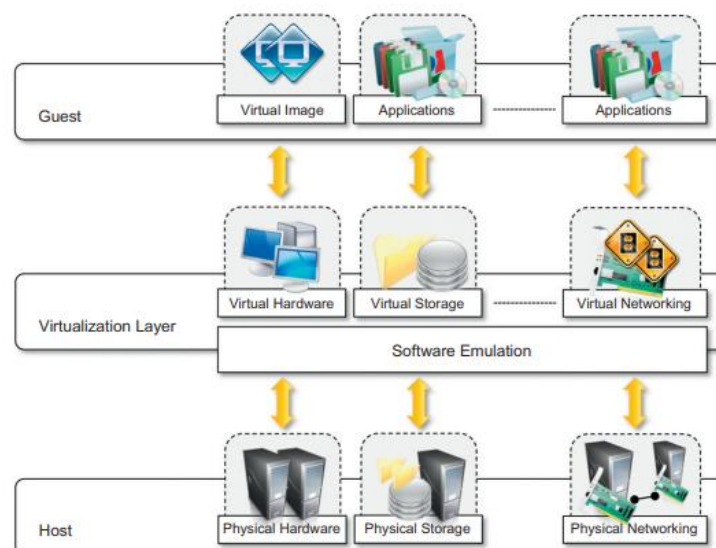


FIGURE 3.1
The virtualization reference model.

1. Structure of the Reference Model

The reference model of full virtualization is divided into **four main layers**:

1. **Hardware Layer** – This is the lowest layer consisting of the actual physical resources like the processor, memory, storage devices, and input/output devices. These are the real components on which virtualization is built.
2. **Virtual Machine Monitor (VMM) or Hypervisor** – This layer sits directly on top of the hardware. The VMM is the most important part of the virtualization model. Its job is to create and manage multiple virtual machines. It allocates hardware resources, ensures security, and makes sure that each VM is isolated from the others. The VMM also traps privileged instructions from the guest operating system and emulates them so that the OS believes it is communicating with real hardware.
3. **Guest Operating Systems** – Above the VMM are the different operating systems installed inside each virtual machine. Since the hypervisor provides complete hardware emulation, the guest OS does not know that it is running in a virtual environment. It behaves as if it has full control over its own hardware, even though resources are being shared.
4. **Applications** – The top layer consists of the applications running inside each guest OS. These applications run without any changes because the OS provides the same environment as it would on a physical machine.

2. Working of the Model

When an application runs inside a virtual machine, it sends requests to the operating system as usual. The OS may issue privileged instructions (for example, accessing memory or input/output devices). Normally, these instructions require direct access to hardware, but in a virtualized system, the **VMM intercepts these instructions**. The VMM then safely executes them on the real hardware and returns the result to the guest OS.

This process ensures that:

- Multiple VMs can share the same physical hardware.
- No VM can interfere with another VM's data or resources.
- The guest OS and applications remain completely unaware that they are running in a virtualized setup.

3. Advantages of Full Virtualization

- **No modification required** → Any operating system can be installed as a guest OS because the VMM provides a complete hardware interface.
- **Strong isolation** → Each VM is completely separate, so a crash in one does not affect the others.
- **Flexibility** → Different operating systems (e.g., Windows, Linux) can run simultaneously on the same machine.
- **Legacy software support** → Old applications can run in a virtual machine without needing changes.

4. Limitations

Although full virtualization is powerful, it also has some drawbacks. The main limitation is performance overhead. Since the VMM has to trap and emulate privileged instructions frequently, execution can be slower compared to running directly on hardware. Modern processors with hardware-assisted virtualization (like Intel VT-x or AMD-V) help reduce this overhead.